MODULE 12

Factor Graphs and LIO-SAM

12.1 From a filter to a graph

SIMPLE APPROACH

Imagine a row of people connected by rubber bands. Each rubber band says "you two should be this far apart." One person is also tied to a post that says "you belong here." If one band pulls a little wrong, everyone wiggles until the pulling is shared out as fairly as possible. A **factor graph** is that picture: the people are the car's positions over time, and the rubber bands are measurements.

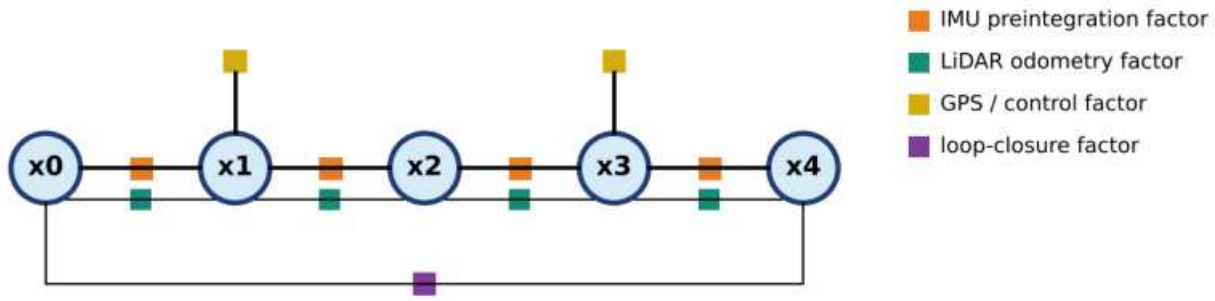


Figure 12.1: A factor graph: circles are unknown states (poses); small squares are measurements (factors) that tie them together.

ENGINEER'S VIEW

A filter (EKF) processes data once and forgets the past. A **smoother** keeps the whole trajectory as unknowns and re-solves when new information arrives — so a loop closure at minute 20 can correct the pose at minute 3. The data structure is a **factor graph**: *variable nodes* $\mathbf{x}_i$ (poses, velocities, IMU biases) and *factor nodes*, each encoding one measurement as a probability that depends on a few variables. Assuming Gaussian noise, finding the most probable trajectory (maximum a posteriori, MAP) is exactly a non-linear weighted least-squares problem — the same mathematics as a survey network adjustment.

KEY EQUATION — MAP estimation becomes least squares

$$\mathbf{X}^* = \arg\max_{\mathbf{X}} \prod_k \phi_k(\mathbf{X}_k) = \arg\min_{\mathbf{X}} \sum_k \left\| \mathbf{h}_k(\mathbf{X}_k) - \mathbf{z}_k \right\|^2_{\mathbf{\Sigma}_k}$$

What every symbol means

Symbol	Meaning	Units
$\mathbf{X}$	all unknowns together (every pose, velocity, bias)	—
$\mathbf{X}^*$	the best (most probable) solution	—
ϕ_k	factor k: a Gaussian probability $\propto \exp(-\frac{1}{2}\|\cdot\|^2_{\mathbf{\Sigma}})$	—
$\mathbf{X}_k$	only the few variables factor k touches	—
$\mathbf{h}_k$	measurement model: what factor k *should* measure given $\mathbf{X}_k$	—
$\mathbf{z}_k$	what it actually measured	—
$\|\mathbf{e}\|^2_{\mathbf{\Sigma}} = \mathbf{e}^\mathsf{T}\mathbf{\Sigma}^{-1}\mathbf{e}$	squared Mahalanobis distance: residuals divided by their uncertainty (weights $= \mathbf{\Sigma}^{-1}$)	—

Taking $-\log$ of a product of Gaussians turns "maximize the product" into "minimize the sum of squares."

12.2 A factor graph you can solve by hand

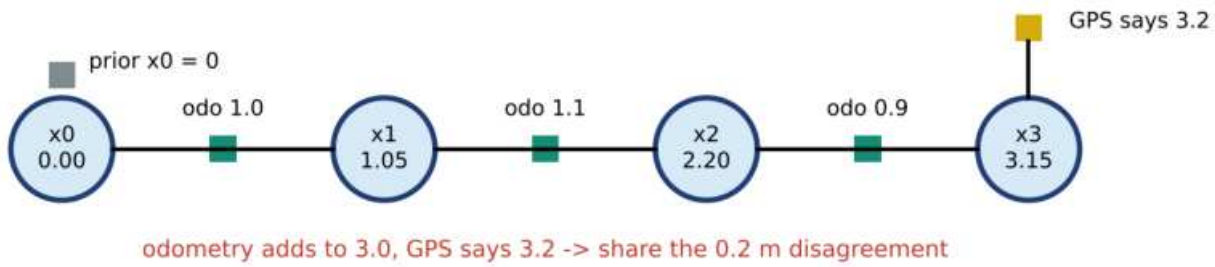


Figure 12.2: Four positions along a straight road, three odometry factors, one GPS factor and a prior.

WORKED EXAMPLE — 1-D pose graph with a GPS misclosure

Given: $x_0 = 0$ (fixed prior). LiDAR odometry: $x_1 - x_0 = 1.0$, $x_2 - x_1 = 1.1$, $x_3 - x_2 = 0.9$ m. GPS: $x_3 = 3.2$ m. All four measurements have the same $\sigma = 0.10$ m (equal weights).

Step 1 — write the residual equations $\mathbf{A}\mathbf{x} = \mathbf{b}$

$$\begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 1.1 \\ 0.9 \\ 3.2 \end{pmatrix}$$

Step 2 — normal equations $\mathbf{A}^\mathsf{T}\mathbf{A}\,\mathbf{x} = \mathbf{A}^\mathsf{T}\mathbf{b}$

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1.0 - 1.1 \\ 1.1 - 0.9 \\ 0.9 + 3.2 \end{pmatrix} = \begin{pmatrix} -0.1 \\ 0.2 \\ 4.1 \end{pmatrix}$$

Step 3 — solve

From row 1: $x_2 = 2x_1 + 0.1$. Row 2: $-x_1 + 2(2x_1 + 0.1) - x_3 = 0.2 \Rightarrow x_3 = 3x_1$. Row 3: $-(2x_1 + 0.1) + 6x_1 = 4.1 \Rightarrow x_1 = 1.05$. Then $x_2 = 2.20$, $x_3 = 3.15$.

Step 4 — residuals

odometry: $1.05 - 1.0 = +0.05$, $1.15 - 1.1 = +0.05$, $0.95 - 0.9 = +0.05$; GPS: $3.15 - 3.2 = -0.05$

Result: $x_1 = 1.05$, $x_2 = 2.20$, $x_3 = 3.15$ m. The 0.20 m misclosure is shared equally by the four equally weighted measurements — the compass/least-squares adjustment of a traverse, done by a factor graph.

ENGINEER'S VIEW

Notice $\mathbf{A}^\mathsf{T}\mathbf{A}$ is *sparse* (mostly zeros): each pose only links to its neighbors. Real graphs have tens of thousands of nodes (LIO-SAM's Amsterdam run: 23 304 nodes, 49 617 factors) but stay sparse, so they solve quickly. **iSAM2** (incremental smoothing and mapping with the Bayes tree) goes further: when a new node arrives it re-solves only the part of the graph that changed, which is how LIO-SAM runs in real time. GTSAM is the open-source library implementing it.

12.3 LIO-SAM, piece by piece

LIO-SAM (Shan, Englot, Meyers, Wang, Ratti & Rus, IROS 2020) is a *tightly-coupled* lidar–inertial odometry and mapping system built on a factor graph. "Tightly coupled" means the IMU and LiDAR are optimized *together* (the IMU biases are estimated from the LiDAR results), rather than the IMU merely supplying a guess.

KEY EQUATION — The LIO-SAM state (paper eq. 1)

$$\mathbf{x} = \left[\mathbf{R}^\mathsf{T},\ \mathbf{p}^\mathsf{T},\ \mathbf{v}^\mathsf{T},\ \mathbf{b}^\mathsf{T}\right]^\mathsf{T}, \qquad \mathbf{T} = [\mathbf{R} \mid \mathbf{p}] \in SE(3)$$

What every symbol means

Symbol	Meaning	Units
$\mathbf{R} \in SO(3)$	attitude (body to world)	—
$\mathbf{p} \in \mathbb{R}^3$	position	m
$\mathbf{v}$	velocity	m/s
$\mathbf{b}$	IMU biases (gyro and accelerometer)	rad/s, m/s^2
$\mathbf{T}$	pose as a rigid transform (Module 6)	—

12.3.1 The four factors

Factor	What it measures	Surveying analog
IMU preintegration	relative motion $\Delta\mathbf{v}_{ij}, \Delta\mathbf{p}_{ij}, \Delta\mathbf{R}_{ij}$ between consecutive states (Module 7)	a measured distance and angle between traverse stations
LiDAR odometry	relative transform $\Delta\mathbf{T}_{i,i+1} = \mathbf{T}_i^{-1}\mathbf{T}_{i+1}$ from scan-to-map matching (Module 11)	a second, independent set of traverse observations
GPS	absolute position of a state (converted to local Cartesian)	tying a station to a control point
Loop closure	relative transform between a new state and an old nearby one	closing the traverse on itself

12.3.2 Keyframes and the sliding window

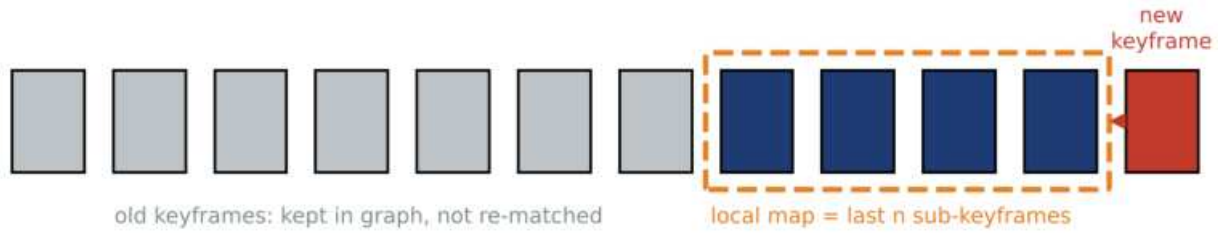


Figure 12.3: Only keyframes enter the graph; a new keyframe is matched to a local map of the last n sub-keyframes.

ENGINEER'S VIEW

Adding every LiDAR frame (10 per second) would make the graph enormous. LIO-SAM adds a new state $\mathbf{x}_{i+1}$ (a **keyframe**) only when the pose changes by more than **1 m** or **10°**; frames in between are discarded. To create a LiDAR odometry factor:

1. extract edge and planar features from the new de-skewed scan (roughness, Module 11);
2. transform the last $n = 25$ keyframes ("sub-keyframes") into the world frame and merge them into a local voxel map $\mathbf{M}_i = \{\mathbf{M}_i^e, \mathbf{M}_i^p\}$, down-sampled at 0.2 m (edges) and 0.4 m (planes);
3. using the IMU-predicted pose $\tilde{\mathbf{T}}_{i+1}$ as the starting guess, find edge/plane correspondences and minimize $\sum d_e + \sum d_p$ by Gauss–Newton;
4. the relative transform to the previous state becomes the factor.

Matching to a small local map, rather than a global one as in LOAM, is the main reason LIO-SAM is fast.

12.3.3 GPS and loop-closure factors

ENGINEER'S VIEW

GPS factor. GPS is converted to a local Cartesian frame; if not hardware-synchronized it is linearly interpolated to the keyframe time. Because lidar–inertial drift grows slowly, a GPS factor is added only when the estimated position covariance is *larger* than the GPS covariance — a sensible "only use control when you need it" rule.

Loop-closure factor. When a new state is added, LIO-SAM searches for earlier states within **15 m**. For a candidate $\mathbf{x}_3$, the new keyframe is matched to the $2m+1$ keyframes around it ($m = 12$), giving $\Delta\mathbf{T}_{3,i+1}$ as a factor. The authors note loop closures are especially valuable for correcting *altitude*, where GPS was poor (errors approaching 100 m without loops in their tests).

12.3.4 What it achieved

End-to-end translation error (m; LIO-SAM in ft in parentheses) — LIO-SAM paper, Table II

Dataset	LOAM	LIOM	LIO-odom	LIO-GPS	LIO-SAM
Campus (1.4 km / 0.9 mi)	192.43	fail	9.44	6.87	**0.12** (0.39 ft)
Park (2.9 km / 1.8 mi)	121.74	34.60	36.36	2.93	**0.04** (0.13 ft)
Amsterdam (19 km / 11.8 mi)	fail	fail	fail	1.21	**0.17** (0.56 ft)

LIO-odom uses only IMU + LiDAR factors; *LIO-GPS* adds GPS; full *LIO-SAM* adds loop closures. Each extra factor type removes another layer of drift. LIO-SAM processed scans in 42–101 ms and ran up to 13× faster than real time.

IMPROVED FROM THE SOURCE MATERIAL

These are research-grade results judged against the start/end point or GPS, on a robot, not survey-grade checkpoint accuracy. A 4–17 cm loop misclosure over kilometers is excellent odometry, but a deliverable still needs calibration, control and *independent checkpoints* (Module 13) before any accuracy can be stated. Also, the paper's eq. (12) writes $\Delta\mathbf{T} = \mathbf{T}_i^{\mathsf{T}}\mathbf{T}_{i+1}$; the correct general form is $\mathbf{T}_i^{-1}\mathbf{T}_{i+1}$.

SURVEYOR'S FIELD TIP

You do not need to write an optimizer, but you should be able to answer: Which factors were active in this dataset? Where were the loop closures? Where was GNSS used? Where did the corridor degrade LiDAR odometry? Good processing software lets you view the trajectory colored by its estimated standard deviation — inspect it before trusting any part of the cloud.

12.4 Building the trajectory: poses, scans, IMU, GNSS and images

SIMPLE APPROACH

Think of a trip recorded as a flip-book. Each page is a **pose**: where the car was and which way it was pointing at one instant. The IMU writes a new page hundreds of times a second, so the flip-book is smooth — but it slowly drifts. The LiDAR compares each new scan with the last ones and corrects the drift, like recognizing the same buildings. GNSS now and then says "this

page is exactly here on Earth." The camera takes pictures along the way, and each picture is glued to the page for the moment it was taken. Flip through all the pages and you have the **trajectory**: the path, with direction, at every instant.

The words

Pose	position (X, Y, Z) *and* orientation (roll, pitch, yaw) at one instant (Module 6)
Trajectory	the continuous sequence of poses over time — the "path with direction" (often delivered as a time-stamped file, e.g. an SBET or a pose table)
Keyframe pose	a pose that is a node in the factor graph (LIO-SAM: every 1 m or 10°)
Interpolated pose	a pose computed *between* keyframes for any time you need: each laser pulse, each image
Image pose	the camera's pose at exposure time (photogrammetrists call it exterior orientation): trajectory pose + camera lever arm and boresight

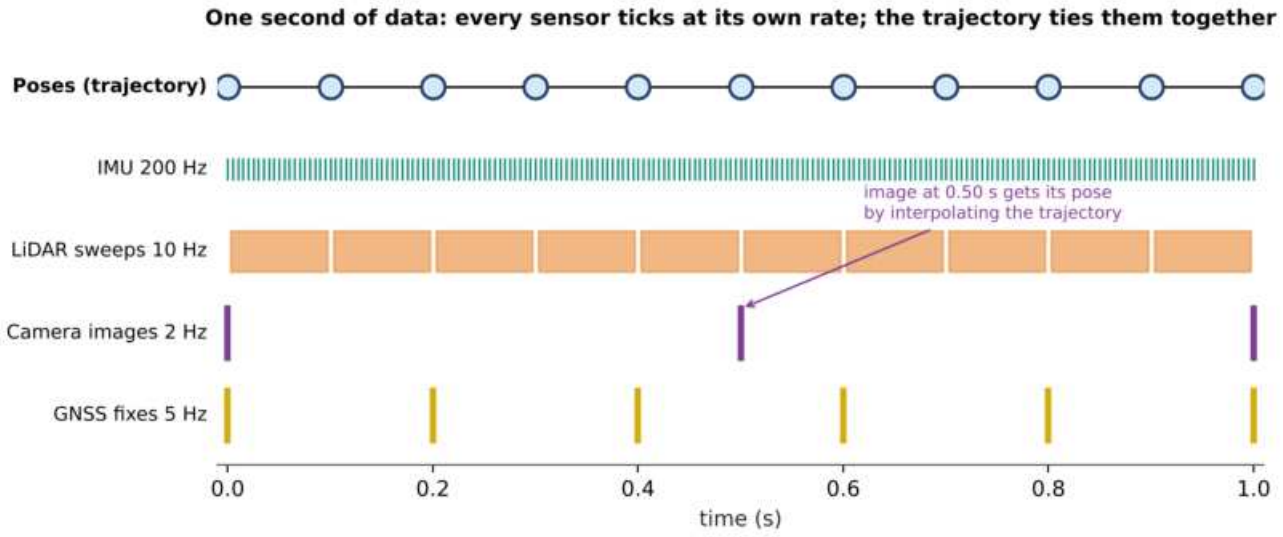


Figure 12.4: One second of data. The IMU ticks 200 times, the LiDAR completes 10 sweeps, GNSS gives 5 fixes and the camera takes 2 images. The trajectory (top row) is the common thread that gives every one of them a pose.

What each sensor contributes

Sensor	Rate	Contributes to the trajectory	Cannot do
IMU	100–1000 Hz	smooth, high-rate motion between everything else; roll and pitch from gravity; de-skews every LiDAR sweep; bridges gaps (preintegration factors)	stay accurate on its own for more than seconds; give absolute position
LiDAR scans	10–20 Hz	relative motion from scan matching (odometry factors); heading and drift control wherever there is geometry; loop closures when places are revisited; the map itself	work in featureless places (corridors, open fields); give absolute coordinates
GNSS	1–20 Hz	absolute position in the datum (GNSS factors); stops long-term drift; RTK makes it centimeter-level; dual antenna adds heading	work in tunnels, under canopy, indoors; supply attitude at high rate
Camera images	1–10 Hz	in LIO-SAM: nothing — images *receive* poses. (Visual–inertial systems also use image features as extra odometry.)	—
Control	a few points	ties the whole trajectory to surveyed coordinates; checkpoints prove it	—

WORKED EXAMPLE — giving a camera image its pose

Given: trajectory poses at $t_1 = 10.00$ s: $E = 100.00$, $N = 200.00$ m, heading $30.0°$; at $t_2 = 10.10$ s: $E = 101.50$, $N = 200.10$ m, heading $31.0°$. A photo is exposed at $t = 10.04$ s.

Step 1 — fraction of the way

$f = (10.04 - 10.00)/(10.10 - 10.00) = 0.4$

Step 2 — position

$E = 100.00 + 0.4(1.50) = 100.60$ m; $N = 200.00 + 0.4(0.10) = 200.04$ m

Step 3 — heading

$30.0 + 0.4(1.0) = 30.4°$ (real software interpolates the full rotation, e.g. quaternion "slerp")

Step 4 — camera offset

apply the camera's lever arm and boresight (Module 9) to move from the IMU to the lens

Result: the image pose is $(100.60,\ 200.04)$, heading $30.4°$. The same interpolation gives every one of the millions of laser pulses its pose.

WORKED EXAMPLE — why the camera needs calibration and timing too

Boresight error $0.1°$, point 20 m away

$20 \times 0.001745 = 0.035$ m — colors land 3.5 cm off their points

Timing error 5 ms at 10 m/s

$10 \times 0.005 = 0.05$ m — colors smear 5 cm along the road

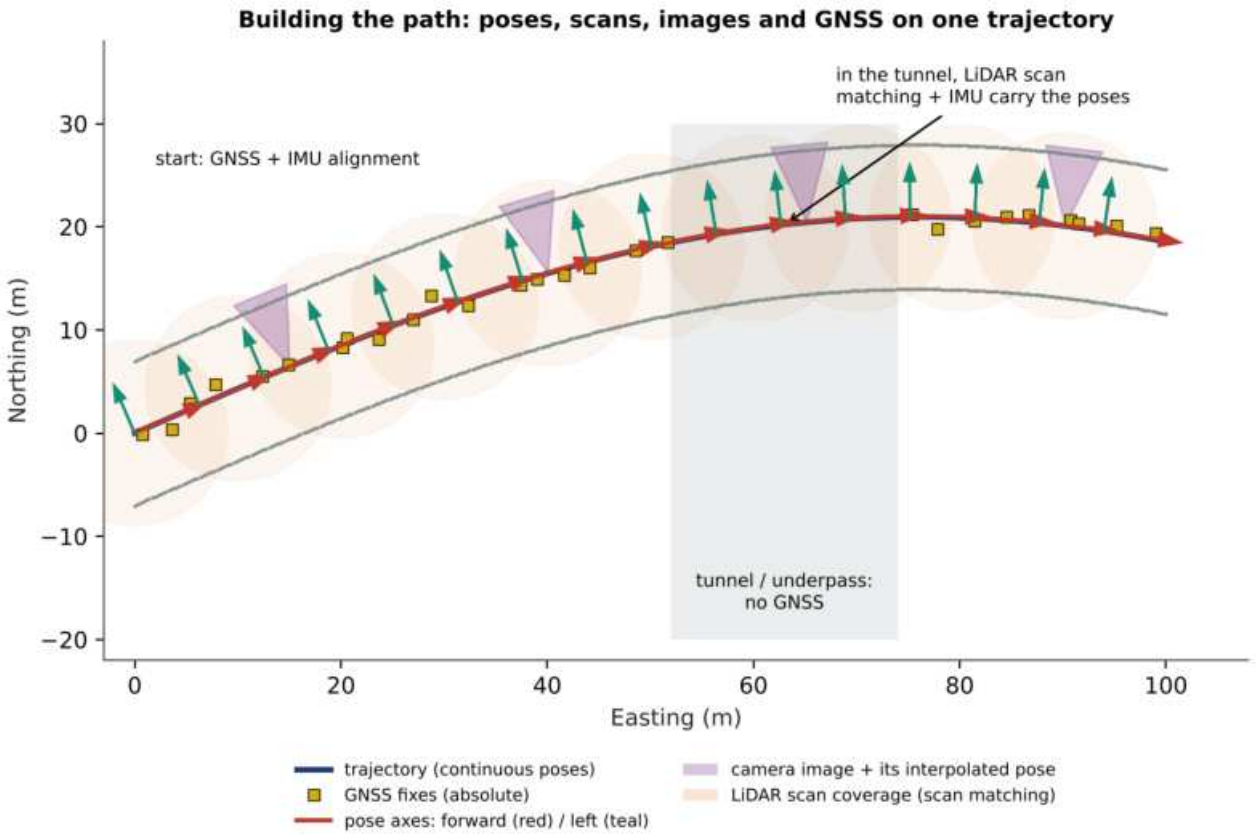


Figure 12.5: The path as built: blue trajectory with pose axes, orange LiDAR scan coverage used for matching, gold GNSS fixes (absent in the tunnel), and purple camera images, each placed with its own interpolated pose.

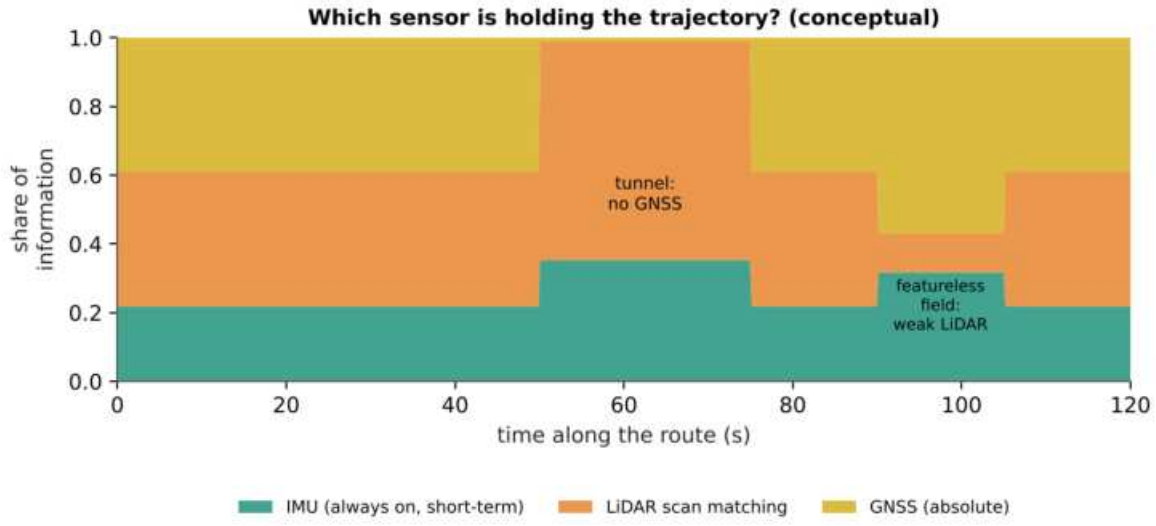


Figure 12.6: Conceptually, the factor graph leans on whichever sensors have information at each moment: LiDAR and IMU carry the tunnel; GNSS and IMU carry the featureless field.

Result: colorized point clouds and photo-measured features inherit every trajectory error *plus* the camera's own calibration and timing errors. Calibrate and synchronize the camera like the LiDAR (Modules 8–9).

SURVEYOR'S FIELD TIP

Deliver and archive the trajectory. Keep the final trajectory (time, position, attitude and their standard deviations), the image exterior orientations, the calibration file and the processing log with the project. When a client later asks "where did this point come from?", the trajectory is the answer — and it is the first thing a reviewer will inspect for drift, gaps and GNSS outages.

TRY IT YOURSELF

12.1 Repeat the 1-D example with GPS saying $x_3 = 3.0$. What happens and why?
12.2 Repeat with GPS $x_3 = 3.2$ but GPS $\sigma = 0.05$ m (weight $4\times$ the others). Solve the weighted normal equations.
12.3 (Simple) In the rubber-band picture, what happens if one band is much stiffer than the rest?

Answers: worked solutions are in Appendix A, "Answers to Try It Yourself," of the Reference Book, starting on page 163.

SOURCES FOR THIS MODULE

Shan *et al.* (2020), LIO-SAM (Tables I–IV, keyframe and loop-closure parameters); Kaess, M. *et al.* (2012), iSAM2, *Int. J. Robotics Research* 31(2); Dellaert & Kaess (2017), Factor graphs for robot perception.